

Backend Low-Level Design

A code-grounded design reference covering services, agent execution, events, persistence, tools, workspaces, APIs, concurrency, security, recovery, and deployment.

Implementation baseline

OpenHands software-agent-sdk commit **cca903c**, dated 31 August 2026. The original documentation is treated as historical context; current source code is the authority where they differ.

Core mental model

Modern OpenHands is an event-sourced agent execution system built around a conversation aggregate. Agent Canvas is the presentation and control layer; Agent Server owns conversation lifecycle and execution; Workspace defines where side effects occur.

Prepared 1 September 2026

Contents

- 1. Scope and architecture evolution
- 2. System component model
- 3. Agent Server lifecycle
- 4. ConversationService
- 5. EventService
- 6. LocalConversation execution loop
- 7. Agent design
- 8. Event and persistence models
- 9. Tool subsystem
- 10. Workspace abstraction
- 11. Plugins, skills, hooks and MCP
- 12. API contract
- 13. Authentication and security
- 14. PubSub, webhooks and telemetry
- 15. Automation service
- 16. Failure and recovery
- 17. Historical mapping
- 18. Design assessment
- 19. Source references

1. Scope and architecture evolution

The linked backend architecture page describes an older generation in which a monolithic backend communicated through a Runtime interface with an Action Execution Server providing Bash, Jupyter, and browser capabilities. The current implementation preserves the intent but reorganizes the boundaries.

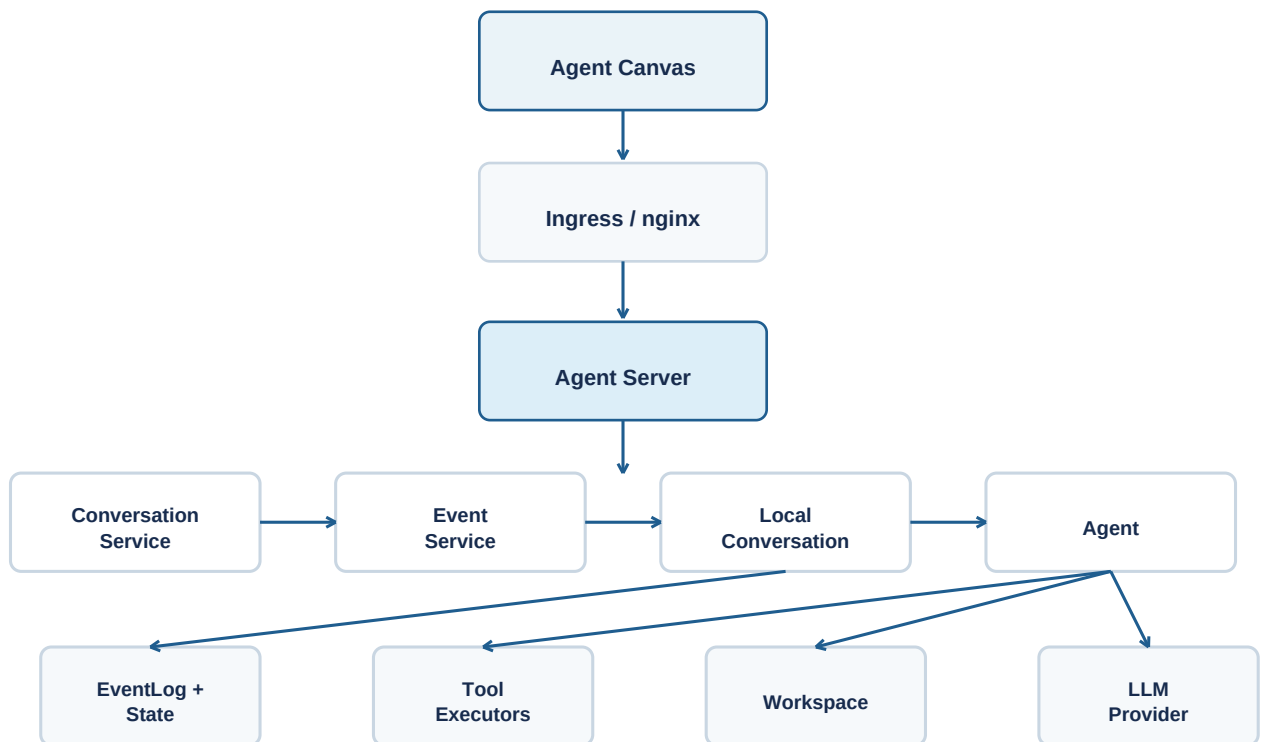
The **OpenHands/OpenHands** repository is now primarily Agent Canvas and local-stack orchestration. The canonical Python backend, agents, events, tools, workspaces, REST API, and TypeScript client live in **OpenHands/software-agent-sdk**. Scheduling lives in a separate Automation repository.

Current equivalent

Agent Canvas -> Agent Server -> ConversationService -> EventService -> LocalConversation -> Agent -> LLM and tools -> Workspace

The historical Runtime and Action Execution Server are no longer a class-for-class match. Workspace, tool executors, and an Agent Server inside isolated environments now cover those responsibilities.

2. System component model



Component	Primary responsibility
Agent Canvas	Renders conversations and controls. It does not execute agent actions.
Ingress	TLS termination and routing for static UI, Agent Server, sockets, and Automation.
Agent Server	FastAPI REST/WebSocket surface and conversation lifecycle.
ConversationService	Process-wide catalog, hydration, leases, eviction, and run capacity.
EventService	Live controller and event broadcaster for one conversation.
LocalConversation	State machine, iteration loop, cancellation, hooks, and budgets.
Agent	Transforms active history into LLM responses and tool actions.
Workspace	Defines the filesystem, shell, Git, process, and network execution boundary.

3. Agent Server lifecycle

The Agent Server is a FastAPI application. Its composition root initializes operational services and mounts public, authenticated, workspace, socket, and OpenAI-compatible routers.

Startup sequence

- Clean stale terminal multiplexer sessions and establish a per-server terminal directory.
- Start telemetry, VS Code integration, and tool preload concurrently.
- Construct ConversationService and BashEventService.
- Start optional retention, eviction, and lease-renewal tasks.
- Mark the process ready only after required dependencies initialize.

Deferred initialization

In deferred mode, liveness endpoints become available while ordinary APIs remain gated. A separately authenticated **POST /api/init** supplies or activates configuration and initializes conversation services.

Shutdown ordering

- Stop accepting new lifecycle work.
- Drain or cancel active runs.
- Stop goal loops, lease renewals, and eviction tasks.
- Close EventServices, tools, MCP sessions, webhook delivery, and telemetry in dependency order.

The ordering preserves tool and MCP availability until agent execution has actually stopped. Validation error handling also removes secret-bearing inputs before returning errors, and telemetry uses route templates instead of raw paths.

Health contract

Endpoint	Meaning
/alive	Process is alive.
/health	Basic server health.
/ready	Required initialization has completed.

Endpoint	Meaning
/server_info	Versions, Git metadata, Python version, usable tools, and capability flags.

4. ConversationService

ConversationService is the process-level owner of stored conversation records and hydrated runtimes. Startup loads lightweight metadata instead of reconstructing every agent, tool, plugin, and MCP connection.

Internal structure

```
ConversationService catalog: conversation_id -> lightweight record event_services: conversation_id ->
EventService lifecycle locks and condition weak per-conversation locks shared run thread pool lease
renewal and eviction tasks credential bindings and webhook configuration
```

A record contains persisted metadata, cached status, metadata and state signatures, the state-file path, and cached ConversationInfo. EventService hydration occurs on access, message delivery, run, subscription, or crash recovery.

Conversation creation

- Validate the parent conversation and require it to use the same workspace.
- Resolve a server-side profile or direct agent configuration.
- Optionally create a Git worktree under the configured conversation worktree root.
- Register Python tool modules, client tool schemas, subagent definitions, plugins, and hooks.
- Persist catalog metadata in meta.json and agent state in base_state.json.
- Create EventService, acquire the lease, subscribe internal consumers, and optionally enqueue the initial message.

Concurrency and ownership

Per-conversation locks serialize lifecycle mutations. A global lifecycle condition coordinates eviction and shutdown. A shared ThreadPoolExecutor caps synchronous fallback runs; the default maximum concurrent run count is 10.

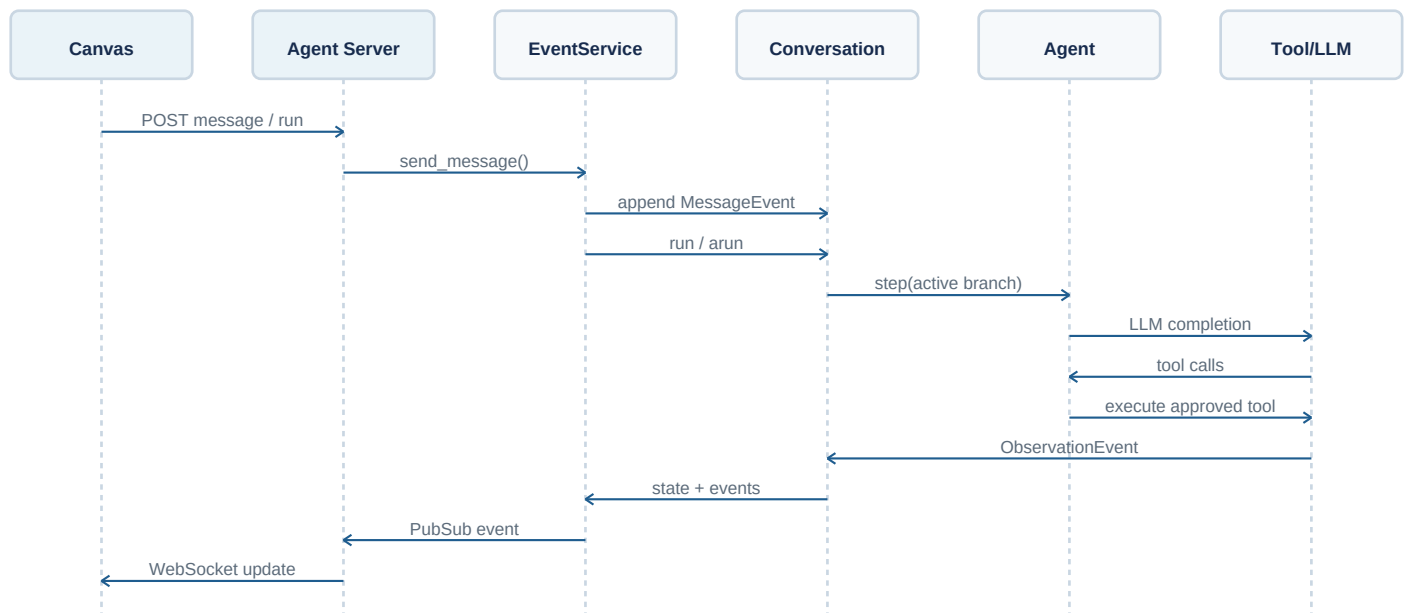
A file-backed ConversationLease identifies owner, generation, and expiry. Central renewal prevents expiry during active use; guarded writes reject stale generations. This avoids split-brain control on shared storage, subject to filesystem-locking limitations such as unreliable flock semantics on some NFS setups.

Idle eviction

Idle EventServices may be evicted while their catalog records and files remain. Running conversations, conversations with external subscribers, and records in active lifecycle operations are not evicted. Future access rehydrates the runtime.

5. EventService

One EventService controls each hydrated conversation. It owns StoredConversation, LocalConversation, PubSub, run and goal-loop tasks, rerun flags, locks, lease state, activity timestamps, and subscriber registrations.



Run behavior

`run()` atomically prevents duplicate runs and returns after scheduling a background task. Native asynchronous execution is preferred. A synchronous agent or conversation runs in the shared executor. Messages arriving during run teardown set a rerun flag so work is not lost at the task-completion boundary.

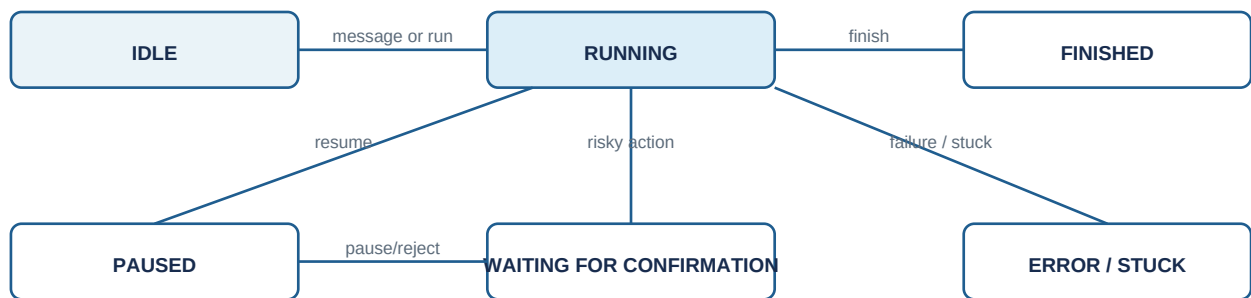
Pause, interrupt, and recovery

- Pause is cooperative and takes effect at a controlled loop boundary.
- Interrupt cancels LLM or tool work, asks executors to interrupt, waits with a bounded timeout, and publishes `PAUSED`.
- Persisted `RUNNING` state after restart is converted to `ERROR`.
- An unmatched `ActionEvent` receives a parented `AgentErrorEvent` so every model tool call has a result.

Subscriptions

A new subscriber immediately receives `ConversationStateUpdateEvent`, can replay all events or events after a timestamp, and then receives live PubSub delivery. WebSocket subscribers opt into `StreamingDeltaEvent`. The first-state send is bounded by a timeout without discarding the subscriber.

6. LocalConversation execution loop



LocalConversation owns ConversationState, Agent or ACPAgent, hooks, plugins, MCP, cancellation, iteration accounting, stuck detection, and budgets. Only user-role messages may enter through `send_message()`. A new message appends MessageEvent, updates HEAD, activates knowledge rules, and can reset FINISHED or STUCK to IDLE.

Iteration order

- Check pause, cancellation, terminal state, iteration limit, and budget.
- Run stuck detection and optional corrective nudge.
- Call `Agent.step()` or `Agent.asteop()`.
- Persist emitted actions and observations.
- Run stop hooks and transition to FINISHED, ERROR, STUCK, PAUSED, or another iteration.

ConversationState uses a fair FIFO lock. The lock is deliberately released while awaiting an LLM network call so messages, snapshots, and interrupts remain responsive. Generation and state checks prevent obsolete results from being committed after superseding input.

7. Agent design

AgentBase is a frozen configuration object containing LLM settings, serializable tool specifications, MCP configuration, skills and repository rules, system prompt, condenser, critic, tool concurrency limit, and private runtime tool instances. The default Agent adds ParallelToolExecutor.

Initialization contract

- Resolve and initialize tool definitions and executors.
- Compose the static system prompt with dynamic workspace, skill, and rule context.
- Append SystemPromptEvent before accepting an ordinary user message.

Agent step algorithm

Stage	Detailed behavior
1. Recovery	Execute unmatched previously approved ActionEvents before requesting another completion.
2. Hook state	Check whether prior hook feedback blocks or redirects execution.
3. Model metadata	Resolve effective context length, tool, vision, and reasoning capabilities.
4. History	Build messages from the cached View of the active event branch.
5. Condensation	Summarize history on overflow or malformed context and rebuild if needed.
6. Completion	Call the LiteLLM-backed provider in chat, responses, or automatic API mode.
7. Classification	Classify tool calls, normal content, reasoning-only output, or empty output.
8. Actions	Create ActionEvents and correlate provider response and tool-call IDs.
9. Policy	Apply security analysis, confirmation policy, critic, and PreToolUse hooks.
10. Execution	Execute approved calls and append typed observations or error events.

Finish semantics

The finish tool is terminal. Calls after finish in the same provider response are discarded. Invalid function calls, empty responses, protocol violations, and context overflow are normally converted into corrective feedback or condensation rather than immediate fatal errors.

LLM layer

LLM configuration covers provider credentials, base URLs, AWS and OpenRouter options, retries, timeouts, token caps, streaming, caching, native tool calling, reasoning behavior, API mode, fallback models, and capability overrides. Runtime model metadata informs condensation and token limits. Per-conversation LLM call context carries tracing and accounting data.

8. Event and persistence models

Events are immutable discriminated Pydantic values with UUID, timestamp, source, parent ID, and a type discriminator. Important families include system prompts, messages, actions, observations, rejection and error events, state updates, streaming deltas, token records, hook records, LLM logs, and condensation markers.

Action and observation correlation

ActionEvent stores thought or reasoning, tool name, typed action, tool-call ID, provider response ID, security risk, critic output, and summary. ObservationEvent references both action ID and tool-call ID. Multiple calls from one provider response are reconstructed as one assistant tool-call message when history is sent back to the model.

On-disk layout

```
// meta.json base_state.json events/ .eventlog.lock event-000000-.json event-000001-.json
observations/
```

The current source writes one indexed JSON file per event. Older text referring to events.jsonl is documentation drift. Event append validates ID uniqueness and parent existence, serializes with a file lock, and can be guarded by the current lease generation.

Event tree

parent_id creates a tree and leaf_event_id acts as HEAD. active_branch() returns the root-to-leaf path. Navigation moves HEAD without deleting sibling branches. Forking copies the entire tree or the path through a chosen event and can reset metrics. Legacy events without parents are interpreted linearly.

Cached View

The View incrementally processes a linear append in $O(k)$, but rebuilds in $O(n)$ after branching, navigation, divergence, or cold restore. During reconstruction it enforces structural properties required by model tool-call protocols.

Mutable state

base_state.json contains status, counters, metrics, active leaf, activated skills, path rules, blocked actions/messages, last user message, tags, secrets registry, agent state, and hook configuration. Public changes autosave; a batching context combines related writes.

9. Tool subsystem

A Tool specification is serializable name plus parameters. A runtime ToolDefinition adds description, JSON schema, typed action and observation models, annotations, and a resolver or executor. Definitions can emit OpenAI, Responses API, and MCP-compatible schemas.

Execution pipeline

```
Validate name -> validate JSON arguments -> build typed Action -> resolve resource locks -> run security and hooks -> execute ToolExecutor -> build typed Observation
```

ParallelToolExecutor

Concurrency defaults to one. Higher limits execute a response batch concurrently. ResourceLockManager applies these rules:

- No declared resources: serialize by tool:.
- Explicit empty resources: no lock.
- Declared resource keys: acquire all corresponding locks in sorted order to avoid deadlock.

Cancellation skips work that has not begun and emits synthetic error observations so the model protocol remains complete. ValueError becomes an agent-visible outcome; unexpected exceptions become internal error events. interrupt() is propagated to active executors.

Tool families

Family	Examples
Execution	Terminal, shell, process, browser-use
Files	Read, editor, apply patch, grep, glob
Repository	Git status, diff, and workspace operations
Agent control	Plan, think, finish, switch LLM
Extension	MCP, skills, workflow, task, delegation, subagents
Client-side	Schema is sent over WebSocket; Canvas performs execution

10. Workspace abstraction

BaseWorkspace defines working-directory resolution, command execution, upload/download, Git changes and diffs, pause/resume, and cleanup.

Implementation	Behavior and trust boundary
LocalWorkspace	Direct host filesystem and process access. Fast, but has Agent Server OS permissions; pause/resume are no-ops.
RemoteWorkspace	Translates operations into Agent Server HTTP calls for a remote or isolated environment.
DockerWorkspace	Starts a prebuilt Agent Server container, mounts selected volumes, injects configuration, health-checks it, and controls it through RemoteWorkspace. Supports real pause/unpause.
Other implementations	Apptainer, cloud, and remote API workspaces can implement the same boundary.

The important modern change is that an isolated Docker workspace commonly runs another Agent Server. It is not merely the historical Action Execution Server.

11. Plugins, skills, hooks and MCP

Plugins normalize a manifest, skills, slash commands, hooks, MCP configuration, and subagents. Claude Code-compatible structures are supported, and commands can be represented as skills. When a Git source is loaded, its resolved commit SHA is persisted for deterministic restoration; source credentials are redacted.

Skills

Skills can come from public, user, project, or plugin scope. Knowledge skills activate through triggers and path rules. Other skills are progressively loaded using an invocation tool so the system prompt does not carry every instruction at all times.

Hooks

Hook	Use
PreToolUse	Inspect, modify, require confirmation, or block an action.
PostToolUse	Inspect an observation and add follow-up behavior.
UserPromptSubmit	React to new user input.
SessionStart / End	Initialize or clean up session-specific behavior.
Stop	Approve completion or deny it and inject continuation feedback.

Hooks may be command, prompt, or agent based and match exact names, wildcards, or regular expressions. A blocked tool produces a rejection observation rather than silently disappearing.

MCP and subagents

MCP tools are dynamically discovered and can be reconciled as server tool lists change. Subagents are registered as AgentDefinitions and exposed through delegation tools while preserving their own prompts and tool configurations.

12. API contract

Area	Representative routes
Health	/alive, /health, /ready, /server_info
Initialization	POST /api/init
Conversations	/api/conversations and ID-scoped start, fork, navigate, run, pause, interrupt, goal, secrets, policies, profile, and plugin routes
Events	/api/conversations/{id}/events for list, search, count, get, batch, message, and confirmation
Workspace	/api/bash, /api/file, /api/git, /api/workspaces
Management	/api/settings, profiles, agent-profiles, provider-connections, llm, mcp, tools, skills, plugins, hooks, subagents
Streaming	/sockets/events/{id}, /sockets/bash-events
Compatibility	A separately mounted OpenAI-compatible router with its own key options

Current routes use /api and /sockets. Older examples without these prefixes should not be treated as implementation authority.

13. Authentication and security

REST API

Authenticated REST routes use the X-Session-API-Key header and require initialized server state. Ordinary API routes do not accept browser cookies as a substitute.

WebSocket

The client sends an auth object containing session_api_key as the first WebSocket message within roughly ten seconds. Invalid authentication closes the socket with a private error code. Query and upgrade-header credentials are legacy paths and are deprecated.

Workspace cookie

An authenticated endpoint can mint oh_workspace_session_key for browser-delivered workspace content. It is HttpOnly, path-scoped, SameSite=None, Secure where appropriate, and Partitioned where supported. Only workspace routes honor it, limiting CSRF exposure.

Secrets and containment

- OH_SECRET_KEY encrypts persisted secrets; without a stable key, sensitive values may be redacted or unrecoverable after restart.
- Resolved paths must remain relative to the configured workspace root, blocking traversal.
- CORS origins and patterns are configurable.
- Validation errors, source URLs, and telemetry paths are sanitized.
- Risk policies, confirmation, security analyzers, and hooks gate side effects.

LocalWorkspace is not a security sandbox. A host-mode Agent Server can execute commands and read or write files with its process permissions. Docker or another isolated workspace is the expected boundary for untrusted tasks.

14. PubSub, webhooks and telemetry

Per-conversation PubSub assigns subscriber UUIDs, caps subscriber count, concurrently delivers events using gather, and isolates subscriber exceptions. It is ephemeral; durable replay comes from EventLog.

Webhook subscribers use bounded queues by count and bytes, batching, retry, and delay. Their failures do not block agent execution. Telemetry is disabled by default and uses an allowlist, bucketed values, and hashed conversation references. It is designed to exclude prompts, files, raw tracebacks, and tool inputs. LLM completion logs and OpenTelemetry/Laminar traces are separate facilities.

15. Automation service

Automation is an optional PostgreSQL-backed FastAPI service. It owns automation definitions, cron evaluation, webhook triggers, run history, dispatch, and workspace lifecycle orchestration. It calls Agent Server to create or run conversations.

Automation decides when work runs. Agent Server decides how an agent execution proceeds.

16. Failure and recovery

Failure	Designed response
Process dies mid-run	Hydrate persisted RUNNING conversation and change it to ERROR.
Action has no result	Append parented AgentErrorEvent to complete the tool-call protocol.
Context overflow	Condense active history, rebuild the view, and retry.
Invalid model call	Add corrective feedback rather than immediately terminate.
Repeated loop	Nudge the model or transition to STUCK.
Iteration or budget limit	Stop additional model execution and publish terminal state.
User interrupt	Cancel LLM/tools, wait for bounded cleanup, and publish PAUSED.
Hook or policy denial	Create rejection feedback or wait for user confirmation.
Webhook/subscriber failure	Retry or isolate without blocking other consumers.
Two servers contend	Lease ownership and generation reject stale writers.

Other resilience mechanisms include stale terminal-session cleanup, ordered shutdown, bounded callback drain before final-state publication, goal-loop generation guards, idle eviction, and centrally renewed leases.

17. Historical mapping

Historical term	Current design
OpenHands Backend	Agent Server
EventStream	EventLog + EventService + PubSub
Agent controller	LocalConversation + Agent
Runtime interface	BaseWorkspace
Docker / remote runtime	DockerWorkspace / RemoteWorkspace
Action Execution Client	Workspace client and ToolExecutor
Action Execution Server	Agent Server within the execution environment
Sandbox plugins	Tools, plugins, skills, hooks, and MCP
Bash session	Terminal tools and Bash event service
Browser environment	Browser-use tool and workspace integration
Jupyter plugin	Not a central first-class component in the inspected current tree

18. Design assessment

Strong decisions

- Append-only typed events provide auditability, branching, and deterministic history reconstruction.
- Lazy hydration keeps startup and memory proportional to active work.
- A shared Agent Server protocol works for host, container, remote, and cloud execution.

- Typed tools and observations make model-provider reconstruction and recovery safer.
- Leases, generation checks, FIFO locking, and bounded concurrency address practical races.

Tradeoffs

- Event files plus mutable base state require careful consistency and migration handling.
- File locking is convenient locally but complicates horizontal scaling and shared filesystems.
- Agent Server inside a workspace simplifies protocols but expands the remote runtime service surface.
- Plugins, hooks, MCP, skills, and client tools create powerful extensibility with substantial lifecycle complexity.

Final characterization

The unit of design is the conversation aggregate: an immutable event tree, mutable execution state, agent configuration, tool and plugin environment, workspace reference, and security policies.
ConversationService manages aggregate lifetimes; EventService makes one aggregate live;
LocalConversation implements its state machine; Agent selects actions; tools create observations;
Workspace contains side effects; Canvas presents and controls the system.

19. Source references

This report distinguishes the older overview from the current source implementation. The source links below are embedded as clickable references.

Reference	Repository location
1. Original backend architecture	docs.openhands.dev / backend architecture
2. Agent Canvas README	OpenHands/OpenHands / README
3. Agent Canvas architecture	OpenHands/OpenHands / docs/architecture.md
4. Self-hosting design	OpenHands/OpenHands / docs/SELF_HOSTING.md
5. Software Agent SDK	OpenHands/software-agent-sdk / README
6. Analyzed SDK commit	software-agent-sdk / commit cca903c
7. Agent Server composition	openhands-agent-server / api.py
8. ConversationService	openhands-agent-server / conversation_service.py
9. EventService	openhands-agent-server / event_service.py
10. LocalConversation	openhands-sdk / local_conversation.py
11. ConversationState	openhands-sdk / conversation/state.py
12. EventLog	openhands-sdk / conversation/event_store.py
13. Agent	openhands-sdk / agent/agent.py
14. Parallel tool execution	openhands-sdk / agent/parallel_executor.py
15. Workspace interface	openhands-sdk / workspace/base.py
16. Docker workspace	openhands-workspace / docker/workspace.py
17. Automation service	OpenHands/automation / README

Version note

OpenHands changes quickly. This document describes the repository structure and source behavior inspected at the stated SDK commit. Routes, storage formats, and service boundaries should be revalidated before implementing integrations against a later revision.